

PEB 遍历的蓝队检测视角（知识点整理）

核心目标：理解蓝队如何检测 `gs:[0x60]` 访问异常，掌握 `NtQueryInformationProcess + ProcessDebugPort` 检测调试器附着的方法，学习三种红队反制策略（拆分遍历逻辑、间接调用、异常处理包裹），建立“攻击-检测-反制”的完整对抗思维。

● 涉及的核心结构体与类型

(一) `pNtQueryInformationProcess`（函数指针类型）

```
typedef NTSTATUS(NTAPI* pNtQueryInformationProcess)(
    HANDLE,                // 目标进程句柄
    PROCESSINFOCLASS,      // 要查询什么信息
    PVOID,                 // 输出缓冲区地址
    ULONG,                 // 缓冲区大小
    PULONG                 // 实际写入字节数（可 NULL）
);
```

用途：动态获取 `NtQueryInformationProcess` 的地址后，强转为该类型调用。

(二) `PROCESSINFOCLASS`（枚举类型）

```
typedef enum _PROCESSINFOCLASS {
    ProcessBasicInformation = 0,
    ProcessDebugPort = 7,           // ← 本课使用的参数（检测调试器）
    ProcessWow64Information = 26,
    ProcessImageFileName = 27,
    ProcessDebugObjectHandle = 30,  // 另一种反调试
    // ...
} PROCESSINFOCLASS;
```

`ProcessDebugPort` 值为 7：查询目标进程的调试端口，非零表示被调试器附着。

(三) `MY_LDR_DATA_TABLE_ENTRY`（自定义结构体）

```
struct MY_LDR_DATA_TABLE_ENTRY {
    LIST_ENTRY InLoadOrderLinks;           // 0x00
    LIST_ENTRY InMemoryOrderLinks;        // 0x10 ← 遍历的链表节点
    LIST_ENTRY InInitializationOrderLinks; // 0x20
    PVOID DllBase;                        // 0x30 ← 模块基址
    PVOID EntryPoint;                     // 0x38
    ULONG SizeOfImage;                     // 0x40
    UNICODE_STRING FullDllName;            // 0x48
    UNICODE_STRING BaseDllName;            // 0x58 ← 模块名
};
```

● 涉及的关键宏与常量

宏/常量	值	作用
<code>ProcessDebugPort</code>	7	<code>NtQueryInformationProcess</code> 的查询类，检测调试器附着
<code>ProcessDebugObjectHandle</code>	30	另一种反调试查询类（获取调试对象句柄）
<code>EXCEPTION_EXECUTE_HANDLER</code>	1	SEH 异常处理宏，表示“执行异常处理代码”
<code>__declspec(noinline)</code>	—	编译器指令，禁止函数内联，保持独立栈帧
<code>__readgsqword(0x60)</code>	—	x64 下读取 TEB 偏移 0x60 处的 PEB 指针

- 完整的利用链（红队反制 vs 蓝队检测）

- (一) 蓝队检测链（检测 PEB 遍历）

蓝队监控手段

- |
 - ├—— 手段 1: 检测 gs:[0x60] 访问频率
 - |├—— Hook 或 ETW 监控 __readgsqword(0x60) 指令
 - |├—— 统计单位时间内访问次数
 - |└—— 超过阈值 → 告警
 - ├—— 手段 2: 检测调试器附着
 - |├—— NtQueryInformationProcess(ProcessDebugPort)
 - |├—— debugPort != 0 → 被调试
 - |└—— 结合其他反调试手段 (IsDebuggerPresent、CheckRemoteDebuggerPresent)
 - └—— 手段 3: 检测异常调用栈
 - |├—— 栈回溯 (Stack Walking)
 - |├—— 检查 Return Address 是否在合法模块内
 - |└—— 发现异常 → 告警

- (二) 红队反制链（三种策略）

反制策略

- |
 - ├—— 策略 1: 拆分遍历逻辑
 - |├—— Step1_GetLdr() 只负责读 PEB 和 Ldr
 - |├—— Step2_FindEntry() 只负责遍历链表
 - |├—— Step3_ExtractBase() 只负责提取基址
 - |└—— 效果: 单个函数不再包含完整的 PEB 访问链
 - ├—— 策略 2: 间接调用
 - |├—— 定义函数指针类型
 - |├—— 用函数指针调用, 而不是直接调用
 - |└—— 效果: 静态分析难以追踪调用链
 - └—— 策略 3: 异常处理包裹
 - |├—— __try / __except 包裹 PEB 访问
 - |├—— 异常被捕获 → 返回 nullptr
 - |└—— 效果: 即使被拦截, 程序也不会崩溃

- 其他必要的扩展知识点

- (一) 蓝队如何检测 gs:[0x60] 访问异常

检测手段:

- ETW (Event Tracing for Windows): 监控用户态 API 调用, 但 gs:[0x60] 是纯内存访问, 不产生 API 调用, ETW 无法直接捕获。
- 硬件断点: 在 gs:[0x60] 指令地址下硬件断点 (需要内核驱动)。
- 内存扫描: 检测进程中是否存在异常的 PEB 遍历代码模式。

➤ 行为分析：统计进程的 PEB 访问频率，超过正常范围则告警。

现实情况：现代 EDR 通常不会直接监控 `gs:[0x60]` 指令，而是通过行为链分析——检测“PEB 遍历 + 动态 API 解析 + 注入”这一整套攻击模式。

(二) NtQueryInformationProcess 的 ProcessDebugPort

函数原型：

```
NTSTATUS NtQueryInformationProcess(
    HANDLE ProcessHandle,           // 目标进程句柄
    PROCESSINFOCLASS ProcessInformationClass, // ProcessDebugPort = 7
    PVOID ProcessInformation,       // 输出缓冲区 (DWORD*)
    ULONG ProcessInformationLength,  // sizeof(DWORD)
    PULONG ReturnLength             // 可 NULL
);
```

返回值规律：

debugPort 值	含义
0	未被调试器附着
非零	被调试器附着，返回的是调试端口号

物理意义：问系统“这个进程有没有调试器？”系统回答“没有 (0)”或“有，调试端口是 0x...”。

(三) SEH（结构化异常处理）基础

```
_try {
    // 可能抛出异常的代码
    pPeb = (void*)__readgsqword(0x60);
}
__except (EXCEPTION_EXECUTE_HANDLER) {
    // 异常处理代码
    std::cerr << "[-] PEB 访问被拦截" << std::endl;
    return nullptr;
}
```

工作原理：

- `_try` 块中的代码执行。
- 如果发生异常（如访问违规），系统查找最近的 `__except` 块。
- `EXCEPTION_EXECUTE_HANDLER` 表示“执行异常处理代码，不继续传播异常”。

用途：在红队开发中，用于兜底——即使 PEB 访问被拦截，程序也不会崩溃，而是优雅退出。

(四) __declspec(noinline) 的作用

```
__declspec(noinline) PEB_LDR_DATA* Step1_GetLdr() {
    PEB* pPeb = (PEB*)__readgsqword(0x60);
    return (PEB_LDR_DATA*)pPeb->Ldr;
}
```

作用：禁止编译器内联该函数。

为什么需要：

- 如果函数被内联，`gs:[0x60]` 访问会分散到各个调用点，增加暴露面。
- 强制 `noinline` 后，每个函数有独立的栈帧，调用链更清晰，但静态分析更难追踪（因

为每个函数只做一小部分)。

(五) 间接调用的原理

```
typedef LIST_ENTRY* (*pFindEntry)(PEB_LDR_DATA*);  
pFindEntry pFunc = RealFindEntry;  
LIST_ENTRY* pEntry = pFunc(pLdr);      // 间接调用
```

原理：

- 直接调用：RealFindEntry(pLdr) → 编译器生成 call RealFindEntry 指令。
- 间接调用：pFunc(pLdr) → 编译器生成 call [pFunc] 指令，通过函数指针跳转。

效果：

- 静态分析（IDA）看到的是 call [pFunc]，无法直接确定目标函数。
- 需要动态调试才能追踪到 RealFindEntry。

(六) 蓝队监控 PEB 访问频率的模拟实现

```
std::atomic<int> g_pegAccessCount{0};  
  
void* SuspiciousGetModuleAddress() {  
    g_pegAccessCount++; // 每次访问 PEB，计数器 +1  
    PEB* pPeb = (PEB*)__readgsqword(0x60);  
    // ...  
}
```

用途：模拟蓝队的“频率监控”逻辑——如果单位时间内 PEB 访问次数超过阈值，则判定为可疑。

现实中的实现：

- 通过 ETW 或内核驱动统计 gs:[0x60] 访问频率。
- 设置阈值（如 100ms 内超过 3 次），超过则告警。

(七) 反调试技术对比表

技术	API	检测目标
IsDebuggerPresent	kernel32!IsDebuggerPresent	调试器附着
CheckRemoteDebuggerPresent	kernel32!CheckRemoteDebuggerPresent	远程调试器
NtQueryInformationProcess (ProcessDebugPort)	ntdll!NtQueryInformationProcess	调试端口
NtQueryInformationProcess (ProcessDebugObjectHandle)	ntdll!NtQueryInformationProcess	调试对象句柄
硬件断点检测	GetThreadContext	调试寄存器 DR0-DR7